INF264 Assingment 1

Introduction:

In this report we describe how we implemented a greedy ID3 decision tree and random forest. We compare our model with Sklearn's own tuned classifiers. We tune our hyperparameters and evaluate our model on a letter dataset, a subset of the Letter Recognition dataset taken from [Sla91]. Our project is organized in three main files, a decision_tree.py file with the implementation of an ID3 algorithm decision tree, random_forest.py file with implementation of the random forest with bagging and max features and a run_experiments.ipynb file where we load the data, pick model, evaluate and compare with Sklearn. We used Numpy for our own implementations and scikit-learn for model choice, evaluation and visualisation.

Use of generative AI:

For this assignment we focused on using AI as a support tool in debugging and researching concepts, rather than writing code. We used it to help us understand error messages, as well as improving our code (for example clarify stopping criteria in the decision tree class). We treated the outputs as advice, and made sure do double check through other sources and discussion. The model design and model implementation is done by us. This use of generative AI allowed us to speed up the troubleshooting, while keeping originality.

Method and implementation:

In the first part of the task, we added the ID3 greedy algorithm. The algorithm is a recursive tree-growing algorithm that, at each node, splits such that it maximizes the information gain with respect to an impurity measure (either gini or entropy). The algorithm is greedy, because at each node it chooses only the currently best split (largest information gain) and then recurses, meaning that it doesn't revisit earlier nodes/choices or search globally for the optimal tree.

In order to do this, we had to implement several class methods:

- fit method: Sets root and converts inputs to correct dtypes and calls build_tree.
- Build tree method: creates a node, checks stopping criteria and finds the best split by calling find_best_split_method, partitions the data and recurses to build two new subtrees (left and right). Otherwise, it sets a leaf with the majority class.

    - Stopping criterias. We create a leaf if any of these criterias are met and the leaf predicts the majority class at that node:
        - The node is pure (all labels are identical)
        - Max depth is reached
        - All feature rows at the node are identical (no further partition possible)
        - The best split have non-positive information gain or split yields an empty child.
- Find best split feature: draws candidate features via feature_subset method (which we added in the next section). Computes median and evaluates the information gain. Returns the best feature index and threshold.
- Calculate information gain: return parent impurity minus weighted child impurity.
- Calculate impurity, calculates entropy or gini of the argument.
- Feature_subset: implemented in the next section. Decides how many features that should be used.
- Predict: traverses the built tree for each row and returns the predicted label. (traverses from root and test x[feature_index] <= threshop until we reach a leaf).
- Print_tree: prints (a readable) output of the tree.

Note that on tie breaking: if two splits yield the same information gain at a node, we keep the first encountered candidate.

The split mechanism:

How we split our continuous features. We subsample features at each node via max_features (if None, then we consider all features). We compute the median for each candidate feature and use that as the split threshold. Then we evaluate the information gain of the resulting binary partition x[feature] <= median vs > median. We keep the feature/threshold with the highest information gain. If we have no valid splits that yield positive gain (or one child would be empty), we stop and create a leaf. Using the median gives a fast, stable and deterministic threshold for continuous features and it is less computationally demanding than f.example evaluating midpoints. This makes the training more efficient and is the reason we chose to use the median as threshold (in accordance with the task).

This means that we made the tree recursively by choosing a feature split that maximizes information gain at each node, by measuring entropy/gini. We have instructed the tree to

create a leaf when the node is pure, when max depth is reached, if all feature-rows are identical or that the best split doesn't give a positive information gain.

Random Forest:

The second part of the task was to implement a random forest. A random forest is a group of decision trees trained on different bootstrap samples (resampling technique, repeatedly picking samples from the data source with replacement, so that some datapoints can be picked more than once for a tree) and with a randomized feature choice at each split (the algorithm randomly selects a subset of features to consider at each split). The trees are combined by majority voting. That is, after every tree has made prediction, the forest combines the outputs by taking a "vote", with the most common prediction being its answer. Bagging ensures that each tree uses a slightly different dataset, hence reduces variance and makes sure that any error in a tree don't align with potential errors in other trees.

The random forest took new parameters in, n_estimator (number of trees in each group), max_features: node-level feature, that select the number of feature at each node. We had to modify our decision tree to take this into account, so we added a new argument max_features and the class method feature_subset in decision_tree.py file, which draws a subset of features at each node. We integrated it into the find best split feature method, so the search is restricted to the sampled candidates and kept median as threshold.
Our random forest have these class methods:

- fit: for each of the n_estimator trees, we draw a bootstrap sample of size n=len(x). We then train new tree on this and stores it in self.trees, with a random subspace components.
- Predict: collect tree predictions into an array with shape (n_estimators, n_samples).

All in all, this means our random forest uses bagging to ensure that each tree that is being "grown" sees a slightly different dataset, therefore lowering total variance. The max_features forces trees to consider a different subset of feature at each split, which will reduce the correlation between the trees and make them more different from eachother. Majority vote aggregates all the different trees, making it a decent classifier.
Our decision tree is deterministic, it gives the same structure every run (given the same data, hyperparameters and seed). That means when max_features = None, we consider all features at each node. If we iterate the candidate list in a fixed order (sorted) and always pick the first feature if there is a tie in information gain, we get a deterministic result. Since we are making a random forest, we shuffle the candidate-feature order and thereby introduce randomnes, even if we have a tie-break. This results in trees changing across runs and that is part of what we use in the random forest. That means, if max_features is None, our tree behaves like a normal decision tree, if we sort our candidates. We control all of this by setting seeds, so our results can be reproduced.

Data analysis

We decided to split our dataset into train test with (80/20) with a random state (for reproduction). We model on the training data and keep the test data untouched till we are testing on the best hyperparams. The reason we choose 80/20 is that we use a 5-fold cross validation, so we use as much training data as possible and try to maintain a big enough chunk of test data. Since we only have 2000 rows of data, we decided to use 5-fold validation to get as much training as possible from the dataset. That way, every datapoint in the training set is used 4 times to train the model, and once as validation, and we kept 400 rows in the test set (a relatively big chunk of the data) in order to reduce the variance of the performance measure (test accuracy).

After splitting our dataset into training and test (80/20), we inspect our training data (and avoid looking at our test data to not get any data leak). We use the "letters.csv" with 2000 rows (1600 rows in the training df). We make a dataframe and print the shape, head and describe our test data. We see that we have 16 features, 6 classes (0-5, or A-F). We assume continuous features, descrete labels and no missing values. The dataset is a subset of the "Sla91 Letter Recognition".

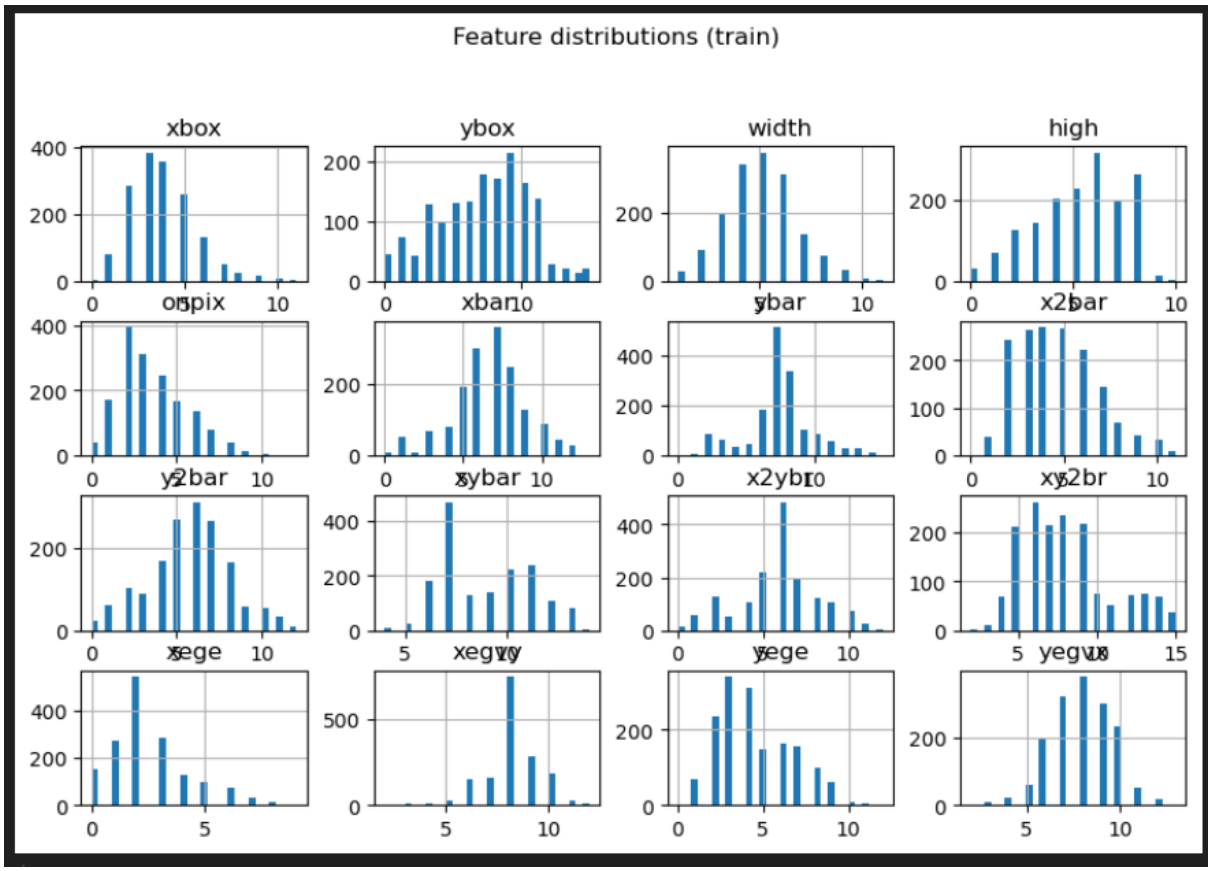
The summary:

```
(1600, 16) (1600,)
   xbox  ybox  width  high  onpix  xbar  ybar  x2bar  y2bar  xybar  x2ybr  \
0   5.0  11.0    6.0   8.0    4.0   3.0   8.0    6.0    6.0   12.0   10.0
1   7.0   9.0    6.0   4.0    5.0   9.0   7.0    3.0    5.0    9.0    4.0
2   2.0   5.0    3.0   3.0    3.0   7.0   7.0    5.0    5.0    6.0    6.0
3   4.0   9.0    4.0   4.0    4.0   8.0   7.0    3.0    4.0    9.0    6.0
4   6.0  11.0    7.0   8.0    4.0   6.0   7.0   11.0   11.0    6.0    5.0

   xy2br  xege  xegvy  yege  yegvx
0   13.0   2.0    9.0   3.0    7.0
1    7.0   7.0    5.0   7.0    8.0
2    6.0   2.0    8.0   6.0   10.0
3    8.0   5.0    7.0   6.0    8.0
4    6.0   3.0    8.0   4.0    8.0
```

	xbox	ybox	width	high	onpix	xbar	ybar	x2bar	y2bar	xybar	x2ybr	xy2br	xege	xegvy	yege	yegvx
count	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000	1600.000000
mean	3.808750	7.004375	4.919375	5.185625	3.493750	6.685000	7.238750	4.630625	5.675000	8.769375	5.820000	8.085000	2.498750	8.083750	4.443125	7.976250
std	1.703008	3.319449	1.694748	2.159649	2.003739	2.255594	2.399551	2.110858	2.357244	2.204648	2.332327	2.841931	1.702369	1.336639	2.163548	1.636264
min	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	4.000000	0.000000	2.000000	0.000000	2.000000	0.000000	2.000000
25%	3.000000	5.000000	4.000000	4.000000	2.000000	5.000000	6.000000	3.000000	4.000000	7.000000	5.000000	6.000000	1.000000	8.000000	3.000000	7.000000
50%	4.000000	7.000000	5.000000	5.500000	3.000000	7.000000	7.000000	4.000000	6.000000	8.000000	6.000000	8.000000	2.000000	8.000000	4.000000	8.000000
75%	5.000000	9.000000	6.000000	7.000000	5.000000	8.000000	8.000000	6.000000	7.000000	11.000000	7.000000	9.000000	3.000000	9.000000	6.000000	9.000000
max	11.000000	15.000000	11.000000	10.000000	12.000000	13.000000	15.000000	11.000000	12.000000	14.000000	12.000000	15.000000	9.000000	12.000000	12.000000	13.000000

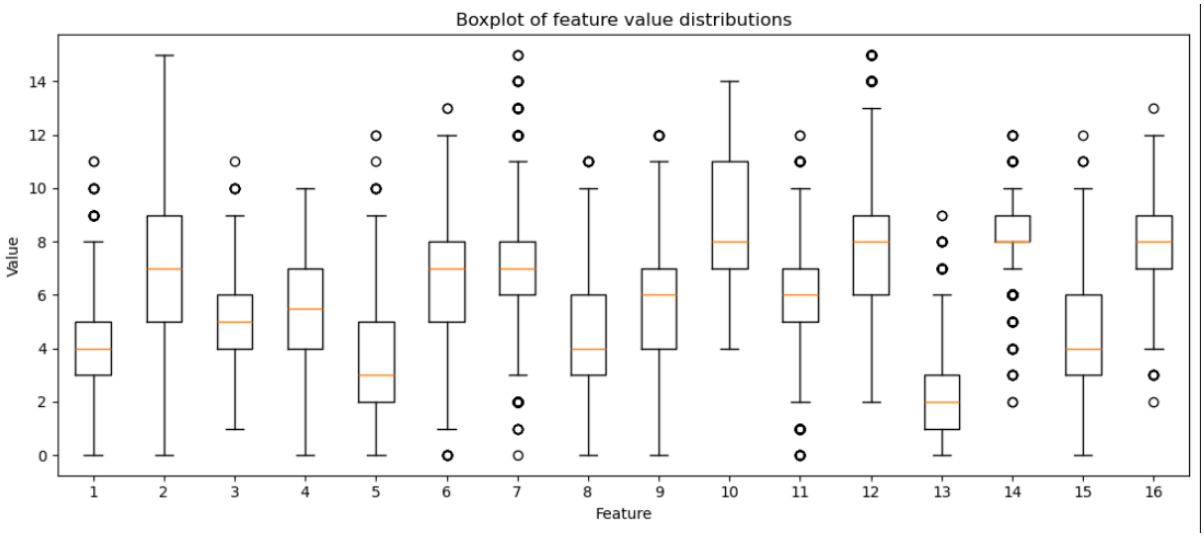
We see that the features vary from 0 to 10 or 15 and there is no need for any ffilling or dealing with incomplete information. Our dataset looks fine.



These graphs show how the different features are distributed, with feature value on the x-axis and the frequency of the value on the y-axis. The histogram shows us how the features are clustered (if they are skewed, spread out etc). We see that there is different scaling along the x-axis (ranges from 0-5 or from 0-15). This may affect the algorithms sensitivity to feature scaling, as there are some features who are more uniformly distributed than others (like xy2br). Perhaps these are the most important features, as they are less likely to introduce bias

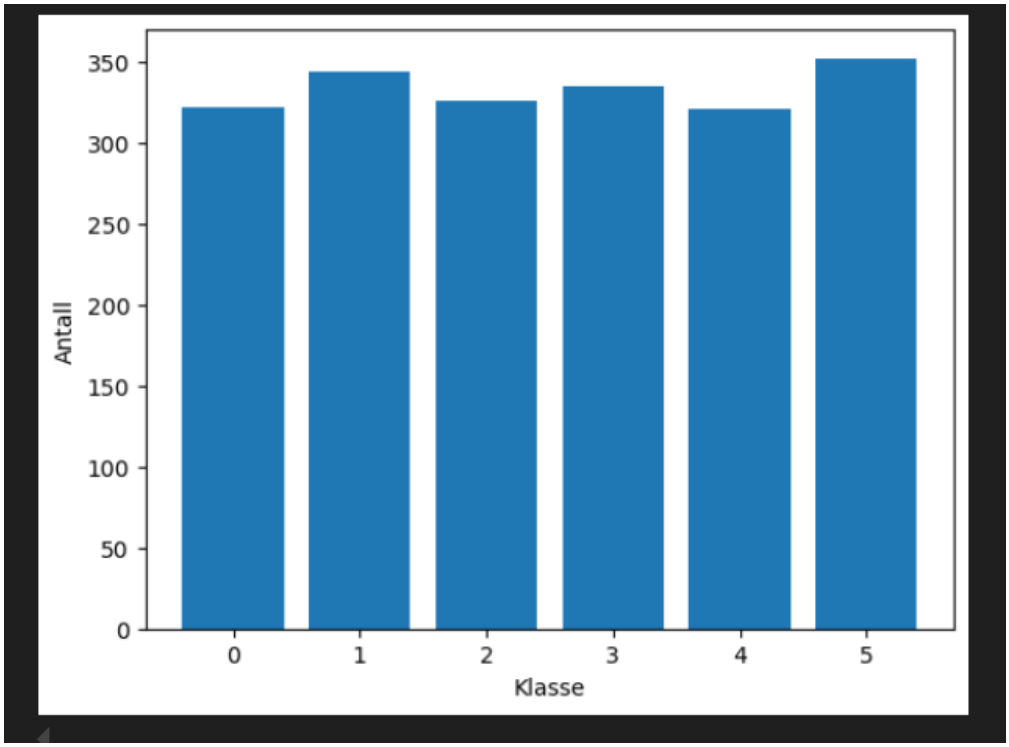
due to scale/sparsity and might therefore be important for our predictions. We'll go further into this in section 4, when we discuss feature importance.

We also see that there are no features with all values in one bin, as that would be useless for the classification (as they wouldn't provide any new information that could be used to decide which label it should have).



This graph shows the distribution of the features in our dataset. It shows IQR, median and some outliers. We can see that some of the features have more outliers than others, which we need to know before making predictions. Can this be due to possible noise in the dataset?

We made a small plot to check how balanced the dataset is. We see that the dataset is quite balanced (almost a uniform distribution) and therefore, we decide to use accuracy as measurement (the grades (A-F) and the total observations of them in the dataset).



Model selection and evaluation

For the model selection and evaluation we divided the dataset into a training set (80%), and a test set(20%) We decided to use 5-fold cross validation, because we have a relatively balanced dataset. To ensure a fair test, we trained all models on the same subsets of the training data. The training dataset was divided into five subsets, and for every subset i, we trained the model on the four other subsets, and tested its accuracy on subset i. For every set of hyperparameters the average accuracy was stored, and the hyperparameters with the highest average accuracy were assigned as the best parameters. We are aware that we use accuracy for this, due to the relatively uniformly distribution on our dataset, and this brings inductive bias toward configurations that performs the best on average over the folds. If a class were asymmetric, another metric (like F1score) would be better suited to use. This process was repeated for the decision tree, as well as the random forest, to find the best hyperparameters. The identical fold across models ensures this fair comparison.

Results

When testing our decision tree on synthetic data, we split the dataset into training data (70%) and validation data (30%). We used the parameters max_depth = None, criterion = “entropy”, and max_features = None (Max features was not a part of the original implementation, but added when creating the random forest classifier). When we fit the model to the training dataset, and predicted for the training dataset, the model received a 100% accuracy, which is to be expected. On the validation dataset, the model received a 93,33% accuracy, also in accordance with our expectations.

```
Training accuracy: 1.0
Validation accuracy: 0.9333333333333333
```

When testing our random forest on a similar set of synthetic data, we tested the parameters n_estimators = 25, max_depth = None, criterion = «entropy» and max_features = «sqrt». The model preformed quite similarly to the decision tree, with a training accuracy of 100% and a test accuracy of 90%.

```
Training accuracy: 1.0
Validation accuracy: 0.9
```

When testing the models on real data, and selecting the best prarameters, we decided to use 5-fold cross validation to get the most possible information from the relatively small dataset. For the decision tree, we did a grid search through max depth from 1 to 15, and the criterion “gini” and “entropy”. Both our model and the sklearn model preformed best using entropy as criterion, but our model performed best with max_depth = 10, while sklearns model preferred max_depth = 15.

```
Our decision trees best params: (10, 'entropy') | CV accuracy: 0.8944
Final test accuracy: 0.9025
Sklearn decision tree best params (15, 'entropy') . CV accuracy:  0.9294
Final test acc sk_learn =  0.9175
```

With the best parameters, our model received a 90,25% accuracy on unseen data, while sklearns model received a 91,75% accuracy. The sklearn model with the best parameters preformed slightly better than our model; The reason for this can be a more in-depth check

for the best split. Our decision tree only split the data by median, while sklearn might check several more splits, which can lead to more information gain per split, and possibly a better decision tree.

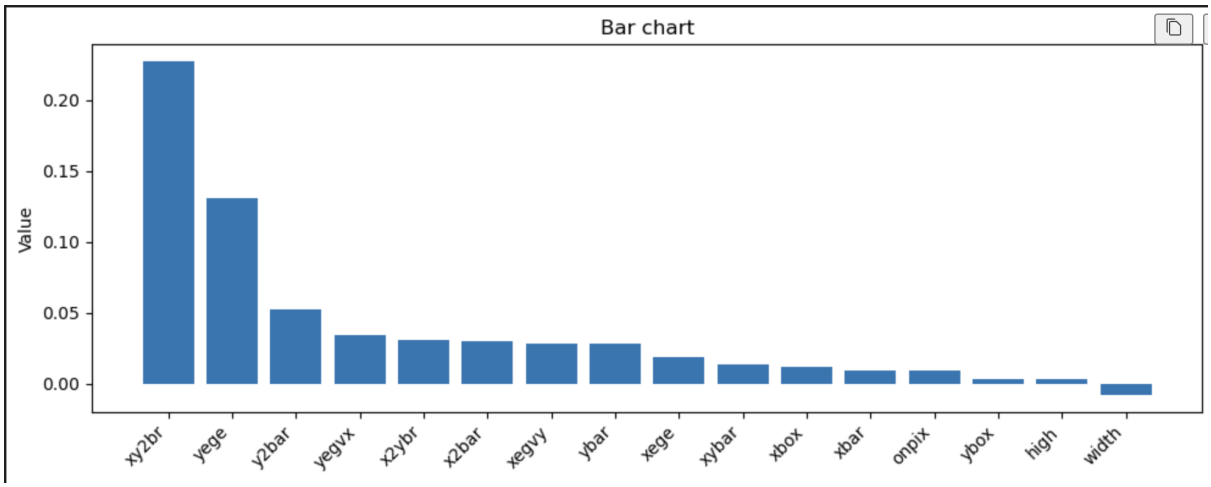
For the random forest evaluation, we also used a grid search with 5-fold cross validation. We searched through the parameters max_depth from 1 to 10 and None, n_estimators from the list [10,30,50,100,200], criterion from entropy and gini, and max_features from log, sqrt, and None.
Our best decision tree (max_depth = 10, n_estimators = 100, criterion = “entropy” and max_features = “log2”) performed with a 96,88% accuracy on the unseen test data, while sklearns model (max_depth = None, n_estimators = 200, criterion = “entropy”, max_features = “log2”) performed a 97,25% accuracy.
The reason for the discrepancy between the models might tie back to the decision trees, where sklearn’s model has a better method for finding the best splits when fitting the different trees.

```
Our Random Forests best params: (10, 100, 'entropy', 'log2') | CV accuracy: 0.9688
Final test accuracy for our model: 0.945
Sklearns Random Forests best params (None, 200, 'entropy', 'log2') . CV accuracy:  0.9681
Final test accuracy for sk_learn model: 0.9725
```

Feature importance

For the feature importance, we implemented a method in the random forest class. To test the feature importance, the model is trained on X_train and y_train, which is 80% of the dataset. To test for the feature importance, we firstly measured a baseline accuracy on the test data, predicting the labels of X_test, and comparing the prediction to y_test.

The next step is to see how the model performs when we shuffle the values of each feature. For every feature i, the method predicts the labels of X_test, with feature i randomly shuffled. We check the accuracy of the prediction against y_test, and compare it to the baseline accuracy. The process is repeated n times (in our case, 30 times). The average difference between the baseline and the recorded accuracy is stored, and returned in a list in order from feature 1 to feature 16 inclusive (or feature 0 to 15 inclusive for computer scientists).



When we correspond each feature to the feature name, and plot in a bar chart in decreasing order, we can see that xy2br and yege seem to be the most important features. The results of the feature importance evaluation can be misleading, because of correlated features - If two important features are correlated, permutating one of the correlated features leaves the other/others to “cover”, which can lead to an underestimated importance score. Furthermore, we see that this aligns with our data analysis, where features that are highly skewed or concentrated in a few bins may yield fewer effective split points, meaning that their importance is very low unless the class boundary is in that specific region. This is in accordance with why the tightly clustered features rank low here.

Division of labour

For this project, we spent most of the time working together at the University, to ensure that both of us got to work with all parts of the project. When we completed the model implementation and model selection, and started on the report, we did some work together, and some work at home. All in all we are satisfied with our division of labour, and both of us contributed to all steps of the project.